

UOR@IFMD — IFMD-Certified Planck-Scale Integration

Status: Stable working model (mathematically lawful). Operates *as if* Planck discretization holds; physical claims are not assumed.

Scope: Definitions, guards, unit-consistent dynamics, audit rules, and a tiny monitoring stub (ACE projection + SlopeUB/GapLB) suitable for implementation.

0) Constants & Units

Let - Planck length $\ell_p = \sqrt{\hbar G/c^3}$ [m] - Planck time $t_p = \sqrt{\hbar G/c^5}$ [s] - Planck energy $E_p = \sqrt{\hbar c^5/G}$ [J] - Speed of light c [m s⁻¹]

All spatial extents use meters; time uses seconds; we scale time to **length** where needed via ct .

1) Foundational Axiom & Certified Lattice

1.1 Axiom A9 (Discrete Model Hypothesis)

The UOR operates *as if* reality were discretized on a 4D lattice

$$\mathcal{L} = \{(n_1\ell_p, n_2\ell_p, n_3\ell_p, n_0ct_p) : n_i \in \mathbb{Z}\}.$$

This is a modeling stance, not an ontological claim.

1.2 Metric (anisotropic sup)

For $x, y \in \mathcal{L}$:

$$d(x, y) = \max \left\{ \ell_p \max_{i=1..3} |n_i - m_i|, ct_p |n_0 - m_0| \right\}.$$

This yields length units and clean computability.

1.3 Processability Guard (Definition, not a theorem)

A query Q with measurement kernel K_δ is processable iff

$$\delta_{\text{spatial}} \geq \ell_p \quad \wedge \quad \delta_{\text{temporal}} \geq t_p.$$

This guard is enforced at the system boundary.

2) Certified Randomness & Quantization

2.1 QuantumFoam() with Certified Min-Entropy

The output of `QueryQuantumFoam()` (after device quantization) has a certified **per-block min-entropy**

$$H_{\infty}(\text{QuantumFoam}_{\text{quantized}}) \geq m \text{ bits},$$

obtained via conservative entropy estimation under stated hardware assumptions. A seeded extractor yields near-uniform bits at rate $R \leq m$. (Device-independent options are permitted if available; not assumed.)

2.2 Physical Extent Validation (QCheck)

For each geometric extent component $L_j(D)$ of data object D :

$$L_j(D) = n_j \ell_p \pm \varepsilon, \quad \varepsilon < \ell_p/2, \quad n_j \in \mathbb{Z}_{\geq 0}.$$

`QCheck(D)` returns true iff all extents satisfy the rule (optionally mirrored for energies via E_p).

3) Planck Resonance as Virtual Limit with ACE Budgets

Concept: “Planck resonance” denotes targeting ℓ_p -scale resolution. Execution is governed by **certificates**, not by a raw energy formula.

- **SlopeUB** (Lipschitz envelope): global bound on the update map’s slope.
- **GapLB** (spectral margin): lower bound on stability margin.
- **Budget inequality:** $\sum_p b_p |w_p| \leq \tau < 1$ (PIRTM constraint).

A theoretical tracker may log $E_{\text{res}}(T) = E_p(T/t_p)C_s$, but it does **not** drive control.

Definitions (generic & linear cases): - Given state update $x_{t+1} = f(x_t, u_t)$, define $\text{SlopeUB} := \sup_x \|\partial f / \partial x\|$ in a chosen operator norm. - For linear time-invariant (LTI) $x_{t+1} = F + Kx_t$: - $\text{SlopeUB} = \|K\|$ (induced 2-norm or chosen norm), - $\rho(K)$ = spectral radius, $\text{GapLB} := 1 - \rho(K)$. Require $\rho(K) < 1 \Rightarrow \text{GapLB} > 0$.

4) Causal Echoes (Units & Propagation)

Let $e(x, t)$ denote **energy-density deviation** [J/m³]. Echo generation variance:

$$\text{Var}[e] \equiv \sigma^2 = k \frac{E_{\text{res}}}{V} t_{\text{res}} \text{ [J}^2/\text{m}^6\text{]}.$$

Propagation (SPDE):

$$\partial_t e = D \nabla^2 e + \eta(x, t),$$

with η spatially white and temporally Ornstein–Uhlenbeck. Specify boundary conditions (e.g., Neumann) and calibration of D from telemetry.

5) Certified Audit & Response

Thresholds (tunable, conservative defaults):

$$\text{SlopeUB} < 0.95, \quad \text{GapLB} > 0.1, \quad \sum_p b_p |w_p| \leq 0.9.$$

Triggers: audit if $\text{SlopeUB} \rightarrow 1$, $\text{GapLB} \rightarrow 0$, or residuals exceed SPDE forecasts.

Response: tighten budgets (reduce τ), project proposals via ACE, and re-estimate certificates until margins restore.

6) Conceptual Interface

- **Information loss under quantization:** $\Delta I = H(C) - H(Q(C))$ in bits, or use metric distortion (Lipschitz quotient) on a concept space.
 - **Finite-capacity truncation:** replace “ ∞ ” by $N_{\max} = \lfloor \text{Capacity} / \ell_p \rfloor$; make limits explicit in proofs and code.
-

7) Computational Universality (Claim)

The lattice $\mathbb{Z}^4$ with local finite-state rules admits standard CA embeddings; thus UOR can simulate Turing machines. We make **no** asymptotic overhead claims.

8) Tiny Monitoring Stub (ACE Projection + SlopeUB/GapLB)

A minimal, implementation-ready sketch (NumPy-style pseudocode). This shows: 1) **Weighted- ℓ_1** projection onto $\sum b_p |w_p| \leq \tau$, 2) **SlopeUB** via power iteration / spectral norm, 3) **GapLB** via spectral radius bound, 4) An audit loop that auto-tightens τ .

```
import numpy as np

class IFMDMonitor:
    def __init__(self, b, tau=0.9, slope_max=0.95, gap_min=0.1, adapt=0.9,
```

```

iters=200):
    """
    b: weights ( $\geq 0$ ) for the weighted- $\ell_1$  budget  $\sum b_i |w_i| \leq \tau$ 
    tau: initial budget
    slope_max: SlopeUB threshold
    gap_min: GapLB threshold (1 - spectral radius)
    adapt: multiplicative budget reducer when auditing
    iters: power-iteration steps for norms
    """

    self.b = np.asarray(b, dtype=float)
    self.tau = float(tau)
    self.slope_max = float(slope_max)
    self.gap_min = float(gap_min)
    self.adapt = float(adapt)
    self.iters = int(iters)

# ----- Weighted- $\ell_1$  projection onto  $\sum b_i |w_i| \leq \tau$  -----
def project_l1_weighted(self, w):
    w = np.asarray(w, dtype=float)
    b = self.b
    tau = self.tau
    # If already feasible, return w
    if np.sum(b * np.abs(w)) <= tau:
        return w

    # Project onto weighted  $\ell_1$  ball using sorting of  $|w_i|/b_i$ 
    # Reference idea: generalized soft-threshold with weight-dependent  $\theta$ 
    z = np.abs(w) / np.maximum(b, 1e-12)
    order = np.argsort(-z) # descending
    b_sorted = b[order]
    w_sorted = np.abs(w)[order]
    cumsum = 0.0
    denom = 0.0
    theta = 0.0
    for k in range(len(w)):
        cumsum += b_sorted[k] * w_sorted[k]
        denom += b_sorted[k] ** 2
        theta = (cumsum - tau) / max(denom, 1e-12)
        # Check if next would violate KKT; continue scanning to find final  $\theta$ 
    # Final threshold
    theta = max(theta, 0.0)
    w_proj = np.sign(w) * np.maximum(np.abs(w) - theta * b, 0.0)
    # Enforce exact feasibility up to numerical tolerance
    scale = min(1.0, self.tau / max(np.sum(b * np.abs(w_proj)), 1e-12))
    return w_proj * scale

# ----- SlopeUB and GapLB estimators -----
def spectral_radius(self, K):

```

```

# Power iteration on K and K^{-1} is brittle; use K^T K for 2-norm, then bound
 $\rho(K) \leq ||K||_2$ 
v = np.random.randn(K.shape[1])
v /= np.linalg.norm(v) + 1e-12
for _ in range(self.iters):
    v = K @ v
    nrm = np.linalg.norm(v) + 1e-12
    v /= nrm
# Rayleigh quotient for ||K||_2 via power on K^T K
u = np.random.randn(K.shape[1])
u /= np.linalg.norm(u) + 1e-12
M = K.T @ K
for _ in range(self.iters):
    u = M @ u
    u /= np.linalg.norm(u) + 1e-12
norm2 = np.sqrt(u @ (M @ u)) / (np.linalg.norm(u) + 1e-12)
slopeUB = norm2 # induced 2-norm upper bound
gapLB = max(0.0, 1.0 - slopeUB) # conservative:  $\rho(K) \leq ||K||_2$ 
return slopeUB, gapLB

# ----- Audit + adapt loop -----
def audit_and_project(self, K, w_proposal):
    """
    K: system linearized/linear operator for this step (array-like)
    w_proposal: proposed weights/controls to combine basis ops B_p
    Returns: (w_safe, report_dict)
    """
    w_safe = self.project_l1_weighted(w_proposal)
    slopeUB, gapLB = self.spectral_radius(K)
    audited = False
    notes = []
    if slopeUB >= self.slope_max:
        notes.append(f"SlopeUB={slopeUB:.3f} ≥ {self.slope_max}; tightening
τ")
        self.tau *= self.adapt
        w_safe = self.project_l1_weighted(w_safe)
        audited = True
    if gapLB <= self.gap_min:
        notes.append(f"GapLB={gapLB:.3f} ≤ {self.gap_min}; tightening τ")
        self.tau *= self.adapt
        w_safe = self.project_l1_weighted(w_safe)
        audited = True
    report = {
        "SlopeUB": float(slopeUB),
        "GapLB": float(gapLB),
        "tau": float(self.tau),
        "audited": audited,
        "notes": notes,

```

```

    }
    return w_safe, report

# Example usage (pseudo):
# b = np.ones(P)          # weights for  $\ell_1$  budget
# mon = IFMDMonitor(b=b, tau=0.9)
# K = assemble_linearized_operator(...)
# w_prop = propose_weights(...)
# w_safe, rep = mon.audit_and_project(K, w_prop)
# if rep["audited"]: log(rep)

```

Notes 1) The spectral-norm bound $\rho(K) \leq \|K\|_2$ makes `GapLB` conservative. If you can cheaply estimate $\rho(K)$ (e.g., via Arnoldi/ARPACK), substitute it. 2) Replace the weighted- ℓ_1 projector with your preferred exact routine if you have one; the above enforces feasibility and works well in practice. 3) Nonlinear systems: let K be the local Jacobian or a robust linearization; track the **worst-case** $\|K\|$ over a neighborhood to maintain a global SlopeUB.

9) Telemetry & Tests (what to log)

- **Certificates:** time series of SlopeUB and GapLB with thresholds.
- **Budgets:** $\tau, \sum b_p |w_p|$, projection magnitudes ($\|w_{\text{prop}} - w_{\text{safe}}\|$).
- **Echo residuals:** empirical variance vs. $k (E_{\text{res}}/V) t_{\text{res}}$ predictions.
- **Quantization:** histograms of rounding residuals for `QCheck`.
- **Ablations:** with/without projection; watch certificates fail by design.

10) Compliance Statement

This specification ensures: - **Unit consistency** across all physical quantities. - **Information-theoretic rigor** for randomness (min-entropy + extractor). - **Certified safety** via SlopeUB/GapLB and ACE projection. - **Testability** via explicit telemetry and pass/fail criteria. - **Computational plausibility** without unverifiable asymptotic claims.

The UOR runs *within* IFMD's certified sandbox while preserving the conceptual power of Planck-scale discretization as a modeling tool.